

تعلم ++C من الصفر إلى الاحتراف

دليل عربي عملي للبرمجة الحديثة وإدارة الذاكرة

إعداد: Manus AI – الإصدار الأول 2026

الفهرس

1	1	بداية صحيحة مع ++C
1	2	الفصل الأول: الأساسيات
1	2.1	أول برنامج والمتغيرات
1	2.2	الشروط والحلقات
2	2.3	تمارين الفصل الأول
2	3	الفصل الثاني: المصفوفات والحاويات
2	3.1	المصفوفة الثابتة
2	3.2	array و vector
3	3.3	تمارين الفصل الثاني
3	4	الفصل الثالث: المؤشرات والمراجع
3	4.1	العنوان والقيمة
3	4.2	المراجع وتمرير القيم
4	4.3	تمارين الفصل الثالث
4	5	الفصل الرابع: إدارة الذاكرة
4	5.1	ما الذي يجب تجنبه؟
4	5.2	المالك الذكي RAII
4	5.3	تمارين الفصل الرابع
5	6	الفصل الخامس: البرمجة الكائنية
5	6.1	الصف والكائن والتغليف
5	6.2	الوراثة والتعدد الشكلي
6	6.3	تمارين الفصل الخامس
6	7	الفصل السادس: التعامل مع الملفات ومشروع عملي
6	7.1	الكتابة والقراءة
6	7.2	مشروع: متتبع درجات
7	7.3	تمارين الفصل السادس
7	8	طريقك إلى الاحتراف

1 بداية صحيحة مع ++C

++C لغة تمنحك تحكماً دقيقاً في الأداء والذاكرة، وتستخدم في الألعاب والأنظمة والبرامج عالية الكفاءة. قد تبدو صريحة أكثر من Python، لكن هذه الصراحة تعلمك كيف يعمل البرنامج فعلاً. الأمثلة هنا تستهدف معيار C++17 أو أحدث. لترجمة ملف اسمه main.cpp باستخدام ++g اكتب g++ -std=c++17 main.cpp -o app ثم شغل ./app ابداً دائماً بتحذيرات مفيدة أثناء التعلّم: g++ -std=c++17 -Wall -Wextra main.cpp -o app < قاعدة السلامة: فضّل الكائنات والحاويات الذكية على الحجز اليدوي للذاكرة؛ استخدم new و delete فقط عندما تفهم سبب حاجتك إليهما.

2 الفصل الأول: الأساسيات

2.1 أول برنامج والمتغيرات

تدخل نقطة التنفيذ عبر الدالة main. تستخدم <iostream> #include لتشغيل الإدخال والإخراج، و std::cout للطباعة و std::cin للقراءة. الفاصلة المنقوطة جزء من صياغة ++C.

```
#include <iostream>
#include <string>

int main() {
    std::string name = "سارة";
    int age = 18;
    double progress = 72.5;
    bool active = true;

    std::cout << "مرحباً " << name << "\n";
    std::cout << "العمر بعد سنة: " << age + 1 << "\n";
    std::cout << "التقدم: " << progress << "%\n";
    std::cout << std::boolalpha << active << "\n";
    return 0;
}
```

اختر النوع وفق البيانات: int للأعداد الصحيحة، و double للعشرية، و char لمحرف واحد، و std::string للنص. اجعل القيم التي لا تتغير const لتمنع تغييرها بالخطأ.

```
const double PI = 3.1415926535;
double radius = 4.0;
double area = PI * radius * radius;
std::cout << area << "\n";
```

2.2 الشروط والحلقات

تحتاج جملة if إلى شرط منطقي. استخدم الأقواس حتى لو كان الجسم قصيراً، فهي تقلل أخطاء التعديل لاحقاً. حلقة for مناسبة للعداد، و while مناسبة للتكرار حتى شرط.

```
#include <iostream>

int main() {
    int score;
    std::cout << "اكتب الدرجة: ";
```

```
std::cin >> score;

if (score >= 90) {
    std::cout << "ممتاز\n";
} else if (score >= 60) {
    std::cout << "ناجح\n";
} else {
    std::cout << "يحتاج إلى مراجعة\n";
}

for (int day = 1; day <= 5; ++day) {
    std::cout << "جلسة تدريب " << day << "\n";
}
}
```

2.3 تمارين الفصل الأول

1. اكتب برنامجاً يقرأ طول وعرض مستطيل ويطبّع المساحة والمحيط.
2. اطبع الأعداد الزوجية من 2 إلى 20 باستخدام حلقة for.
3. اقرأ درجة حرارة واطبع «بارد» أو «معتدل» أو «حار» بثلاثة نطاقات واضحة.

3 الفصل الثاني: المصفوفات والحاويات

3.1 المصفوفة الثابتة

المصفوفة التقليدية تحفظ عدداً ثابتاً من العناصر المتجاورة. فهرس أول عنصر هو صفر، والممرور خارج الحدود خطأ خطير لا يضمن المترجم منعه. لذلك افحص الحدود دائماً أو استخدم حاويات معيارية.

```
#include <iostream>

int main() {
    int grades[5] = {88, 72, 95, 66, 81};
    int total = 0;
    for (int i = 0; i < 5; ++i) {
        total += grades[i];
    }
    std::cout << "المتوسط: " << total / 5.0 << "\n";
}
```

3.2 array و vector

std::vector قائمة ديناميكية آمنة وأسهل استخداماً من مصفوفة محجوزة يدوياً. استخدم push_back للإضافة و size للعدد. أما std::array فتعطي حجماً ثابتاً مع واجهة أكثر أماناً من المصفوفة القديمة.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<std::string> tasks = {"قراءة", "تدريب"};
```

```
tasks.push_back("مشروع");

for (const std::string& task : tasks) {
    std::cout << "- " << task << "\n";
}
std::cout << "عدد المهام: " << tasks.size() << "\n";
}
```

3.3 تمارين الفصل الثاني

1. خزن خمس درجات في مصفوفة واحسب أكبرها وأصغرها.
2. أنشئ `vector<int>` وأضف عشرة أعداد ثم احسب مجموع الأعداد الزوجية.
3. اكتب حلقة تتأكد من أن فهرس المستخدم يقع بين صفر و 1 - `vector.size()` قبل القراءة.

4 الفصل الثالث: المؤشرات والمراجع

4.1 العنوان والقيمة

المؤشر متغير يخزن عنواناً في الذاكرة. الرمز & يأخذ عنوان متغير، والرمز * يصل إلى القيمة الموجودة في العنوان. ابدأ المؤشر دائماً بـ `nullptr` إن لم يكن يشير إلى شيء صالح، ولا تفك مرجع مؤشر فارغ.

```
#include <iostream>

int main() {
    int score = 80;
    int* score_ptr = &score;

    std::cout << "القيمة: " << *score_ptr << "\n";
    *score_ptr = 95;
    std::cout << "القيمة الجديدة: " << score << "\n";
}
```

4.2 المراجع وتمرير القيم

المرجع & اسم آخر للكائن، وهو غالباً أبسط وأمن من المؤشر حين تريد تغيير قيمة مرسلة إلى دالة. مَرّر الكائنات الكبيرة كمرجع ثابت `const T&` عندما تحتاج القراءة فقط.

```
#include <iostream>
#include <string>

void add_point(int& points) {
    ++points;
}

void print_name(const std::string& name) {
    std::cout << name << "\n";
}

int main() {
```

```
int points = 10;
add_point(points);
print_name("ليان");
std::cout << points << "\n";
}
```

4.3 تمارين الفصل الثالث

1. اكتب دالة swap_numbers تتبادل قيمتي عددين باستخدام المراجع.
2. أنشئ مؤشراً يشير إلى أول عنصر في مصفوفة ثم اطبع القيمة الثانية باستخدامه.
3. عدّل دالة تأخذ نصاً طويلاً لتستقبل const std::string& بدلاً من نسخة كاملة.

5 الفصل الرابع: إدارة الذاكرة

5.1 ما الذي يجب تجنبه؟

الحجز اليدوي بـ new يحتاج دائماً إلى delete، وإلا يحدث تسريب ذاكرة. كما أن استخدام مؤشر بعد delete ينتج مؤشراً متديلاً. لا تكتب هذا النمط في برامجك الحديثة إلا عند تمرين تعليمي أو واجهة قديمة تتطلبه.

```
// مثال تعليمي فقط: تجنب هذا الأسلوب في المشاريع الجديدة
int* number = new int(42);
std::cout << *number << "\n";
delete number;
number = nullptr;
```

5.2 المالك الذكي RAII

std::unique_ptr يملك كائناً واحداً ويحرره تلقائياً عند الخروج من النطاق. هذا تطبيق لمبدأ RAII: المورد يتبع عمر الكائن. استخدم std::make_unique بدلاً من new مباشرة.

```
#include <iostream>
#include <memory>
#include <string>

struct Note {
    std::string text;
    explicit Note(std::string value) : text(std::move(value)) {}
};

int main() {
    auto note = std::make_unique<Note>("بأمان ++C أتعلم");
    std::cout << note->text << "\n";
} // تلقائياً هنا Note يتم تحرير
```

5.3 تمارين الفصل الرابع

1. فسّر لماذا يعد المؤشر المتدلي خطيراً، ثم اذكر طريقتين لتجنبه.
2. استبدل مؤشراً عادياً لكائن std::string بـ std::unique_ptr.
3. ابحث في توثيق المكتبة القياسية عن الفرق بين unique_ptr و shared_ptr، ثم اكتب مثال استخدام واقعي لكل واحد.

6 الفصل الخامس: البرمجة الكائنية

6.1 الصنف والكائن والتغليف

الصنف يجمع البيانات والدوال التي تعمل عليها. اجعل بيانات الكائن الخاصة `private`، ثم وفر عمليات واضحة تضمن صحة الحالة. لا تجعل كل حقل عاماً لمجرد تقليل الأسطر.

```
#include <iostream>
#include <string>

class Course {
private:
    std::string title;
    int lessons;
    int completed = 0;

public:
    Course(std::string course_title, int lesson_count)
        : title(std::move(course_title)), lessons(lesson_count) {}

    void complete_lesson() {
        if (completed < lessons) ++completed;
    }

    double progress() const {
        return lessons == 0 ? 0.0 : 100.0 * completed / lessons;
    }

    void print() const {
        std::cout << title << ": " << progress() << "%\n";
    }
};

int main() {
    Course cpp("C++", 10);
    cpp.complete_lesson();
    cpp.print();
}
```

6.2 الوراثة والتعدد الشكلي

استخدم الوراثة عندما يكون الصنف الجديد «نوعاً من» الصنف العام. اجعل الدالة افتراضية `virtual` إذا كنت تحتاج السلوك المتخصص عبر مؤشر أو مرجع للصنف الأب، وأضف مدمراً افتراضياً في الأصناف التي ستستخدم تعدد الأشكال.

```
#include <iostream>
#include <memory>
#include <vector>

class Lesson {
public:
```

```

virtual ~Lesson() = default;
virtual void study() const { std::cout << "اقرأ الدرس\n"; }
};

class CodingLesson : public Lesson {
public:
    void study() const override { std::cout << "اكتب المثال وشغله\n"; }
};

int main() {
    std::vector<std::unique_ptr<Lesson>> lessons;
    lessons.push_back(std::make_unique<CodingLesson>());
    for (const auto& lesson : lessons) lesson->study();
}

```

6.3 تمارين الفصل الخامس

1. أنشئ صنف BankAccount برصيد خاص وطرائق إيداع وسحب تمنع الرصيد السالب.
2. أنشئ صنف Book فيه عنوان ومؤلف، ثم أضف دالة print_info ثابتة القراءة const.
3. صمّم Shape افتراضياً و Circle و Rectangle متخصصين، مع دالة مساحة مشتركة.

7 الفصل السادس: التعامل مع الملفات ومشروع عملي

7.1 الكتابة والقراءة

تستخدم std::ofstream للكتابة و std::ifstream للقراءة. تحقق دائماً من نجاح فتح الملف قبل استعماله. يقرأ std::getline السطر كاملاً، ولهذا هو مناسب للنصوص التي تحتوي مسافات.

```

#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ofstream out("notes.txt");
    if (!out) { std::cerr << "تعذر فتح الملف للكتابة\n"; return 1; }
    out << "تعلمت classes\n" << "تعلمت vector\n";
    out.close();

    std::ifstream in("notes.txt");
    std::string line;
    while (std::getline(in, line)) {
        std::cout << line << "\n";
    }
}

```

7.2 مشروع: متتبع درجات

المشروع التالي يجمع vector والدوال والملفات. يحتفظ بالدرجات في ملف نصي بسيط؛ يمكنك تطويره لاحقاً لإضافة أسماء الطلاب أو فرز النتائج.


```

#include <fstream>
#include <iostream>
#include <numeric>
#include <vector>

double average(const std::vector<int>& grades) {
    if (grades.empty()) return 0.0;
    return static_cast<double>(std::accumulate(grades.begin(), grades.end(), 0)) /
grades.size();
}

int main() {
    std::vector<int> grades;
    int grade;
    std::cout << "\nأدخل الدرجات من 0 إلى 100، واكتب 1- للانهاء";

    while (std::cin >> grade && grade != -1) {
        if (grade >= 0 && grade <= 100) grades.push_back(grade);
        else std::cout << "\nدرجة غير صحيحة";
    }

    std::ofstream file("grades.txt");
    for (int value : grades) file << value << "\n";
    std::cout << "المتوسط: " << grades.size() << ", عدد الدرجات: " <<
average(grades) << "\n";
}

```

7.3 تمارين الفصل السادس

1. طوّر متتبع الدرجات ليقرأ الدرجات المخزنة عند بداية تشغيله.
2. أضف دالة تحسب أعلى درجة وأدنى درجة بدون تكرار حلقة في main.
3. ابنِ دفتر جهات اتصال صغير يحفظ الاسم والهاتف في ملف، ثم يعرضه في التشغيل التالي.

8 طريقك إلى الاحتراف

أكملت أساساً قوياً: التحكم بالتدفق، الحاويات، المراجع، الذاكرة الآمنة، الأصناف، والملفات. طور هذا الأساس بتعلم الخوارزميات، القوالب templates، الاختبارات، وأداة بناء مثل CMake. اكتب مشروعاً صغيراً متكاملًا، ترجم مع التحذيرات، واقراً كل تحذير قبل أن تتجاهله.

< الاحتراف في ++C لا يعني استعمال كل ميزة؛ يعني اختيار أبسط ميزة آمنة وصريحة تحقق المطلوب.